

Letter Frequency Attack on Additive Cipher

15. Write a C program that can perform a letter frequency attack on an additive cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

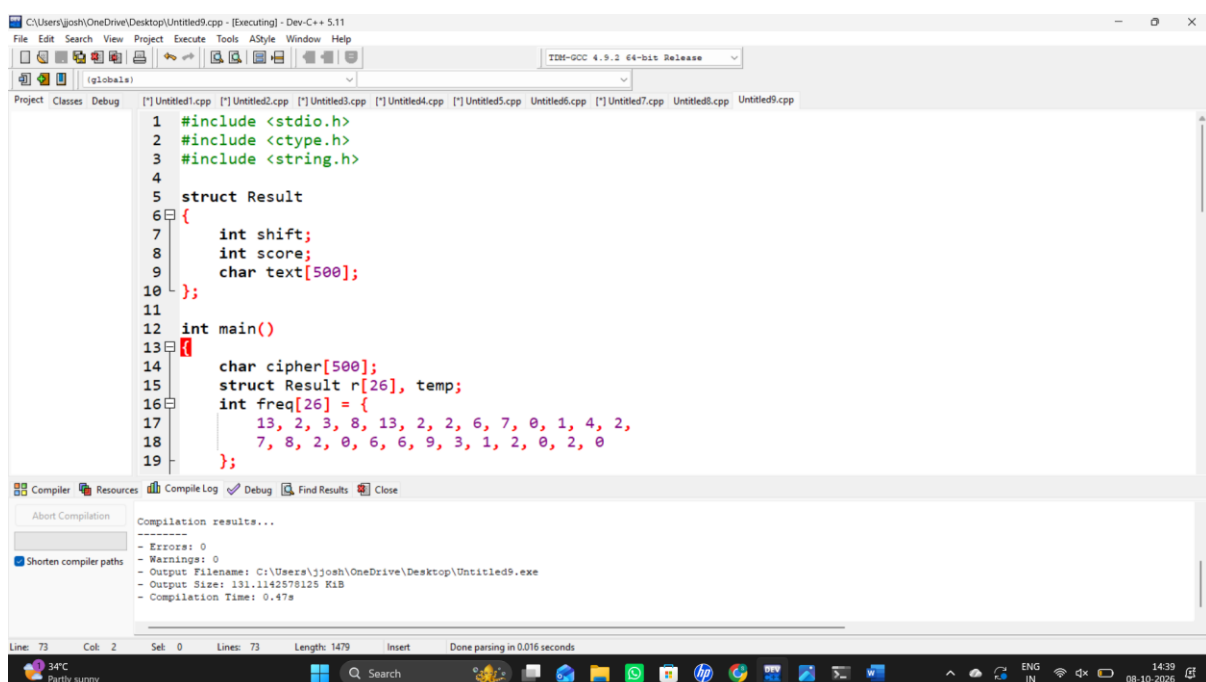
Aim

To write a C program that performs a letter frequency attack on an additive cipher and displays possible plaintexts in order of likelihood.

Algorithm

1. Read the ciphertext.
2. Count the frequency of each alphabet letter.
3. Try all possible Caesar shifts from 0 to 25.
4. Decrypt the ciphertext using each shift.
5. Calculate a simple likelihood score based on common English letters.
6. Store the possible plaintexts and their scores.
7. Sort the results according to the score.
8. Display the top 10 possible plaintexts.

Program



```
1 #include <stdio.h>
2 #include <ctype.h>
3 #include <string.h>
4
5 struct Result
6 {
7     int shift;
8     int score;
9     char text[500];
10 };
11
12 int main()
13 {
14     char cipher[500];
15     struct Result r[26], temp;
16     int freq[26] = {
17         13, 2, 3, 8, 13, 2, 2, 6, 7, 0, 1, 4, 2,
18         7, 8, 2, 0, 6, 6, 9, 3, 1, 2, 0, 2, 0
19     };
20
21     // Read ciphertext
22     printf("Enter ciphertext: ");
23     fgets(cipher, 500, stdin);
24
25     // Process each shift from 0 to 25
26     for (int i = 0; i < 26; i++)
27     {
28         // Decrypt using shift i
29         for (int j = 0; j < strlen(cipher); j++)
30             cipher[j] = (cipher[j] - i + 26) % 26;
31
32         // Calculate likelihood score
33         int score = 0;
34         for (int k = 0; k < strlen(cipher); k++)
35             score += freq[cipher[k] - 'A'];
36
37         // Store result
38         r[i].shift = i;
39         r[i].score = score;
40         strcpy(r[i].text, cipher);
41     }
42
43     // Sort results by score
44     for (int i = 0; i < 26; i++)
45         for (int j = i + 1; j < 26; j++)
46             if (r[i].score < r[j].score)
47                 swap(r[i], r[j]);
48
49     // Display top 10 results
50     printf("Top 10 possible plaintexts:\n");
51     for (int i = 0; i < 10; i++)
52     {
53         printf("Shift: %d, Score: %d, Plaintext: %s\n",
54             r[i].shift, r[i].score, r[i].text);
55     }
56 }
```

Compilation results...

- Errors: 0
- Warnings: 0
- Output Filename: C:\Users\josh\OneDrive\Desktop\Untitled9.exe
- Output Size: 131.1142578125 KiB
- Compilation Time: 0.47s

Input/Output

```
C:\Users\josh\OneDrive\Desktop >
Enter ciphertext: KHOOR ZRUOG
Top 10 Possible Plaintexts:
1. Shift = 14 Score = 78
   WTAAD LDGAS
2. Shift = 6 Score = 74
   EBIIIL TLOIA
3. Shift = 10 Score = 69
   AXEEH PHKEW
4. Shift = 3 Score = 63
   HELLO WORLD
5. Shift = 7 Score = 54
   DAHHK SKNHZ
6. Shift = 13 Score = 52
   XUBBE MEHBT
7. Shift = 17 Score = 52
   TQXXA IADXP
8. Shift = 21 Score = 52
   PMTTW EMZTL
9. Shift = 25 Score = 49
   LIPPS ASVPH
10. Shift = 0 Score = 48
    KHOOR ZRUOG

-----
Process exited after 14.65 seconds with return value 0
Press any key to continue . . .
```

Result

Thus, the program successfully performed a letter frequency attack on the additive cipher and displayed the top 10 possible plaintexts in approximate order of likelihood